Where 4-Bit Quantization Breaks Small Tool-Calling Agents, and Which Repairs Fix It

CMPE 295A · Workbook 1 · October 9, 2026

Ekant Kapgate **Vritika Malhotra** **Yashashav Devalapalli Kamalraj**

Advisor: Dr. Bertin Cordova-Diba

Small agents ship at 4 bits, and the damage hides

8.0 → 2.5 GB

Qwen3-4B weights, 16-bit vs 4-bit Q4_K_M
Taheri et al. [19]

19.5% → 38.3%

Calls to tools that don't exist, 16-bit → 4-bit; task score shows no significant change
τ^2-bench, Gemma-4-31B, Jang et al. [21]

62.3% → 16.8%

Qwen3-0.6B on BFCL single-turn at Q4_K_M
Taheri et al. [19]

Aggregate scores hide where 4-bit breaks small agents.

Three repairs, never compared

Repair	What it changes	Judged by	Gap
R1 Restrict output (grammar)	Decoder masks invalid tokens	Tool-call validity, speed	Can’t fix valid-but-wrong calls
R2 Add bits	Precision of some layers or steps	Perplexity, continuation scoring, ALFWorld/WebShop	Step vs layer vs uniform untested on tool calling
R3 Retrain (QAD)	Weights, same 4-bit format	Aggregate scores	No train-longer control

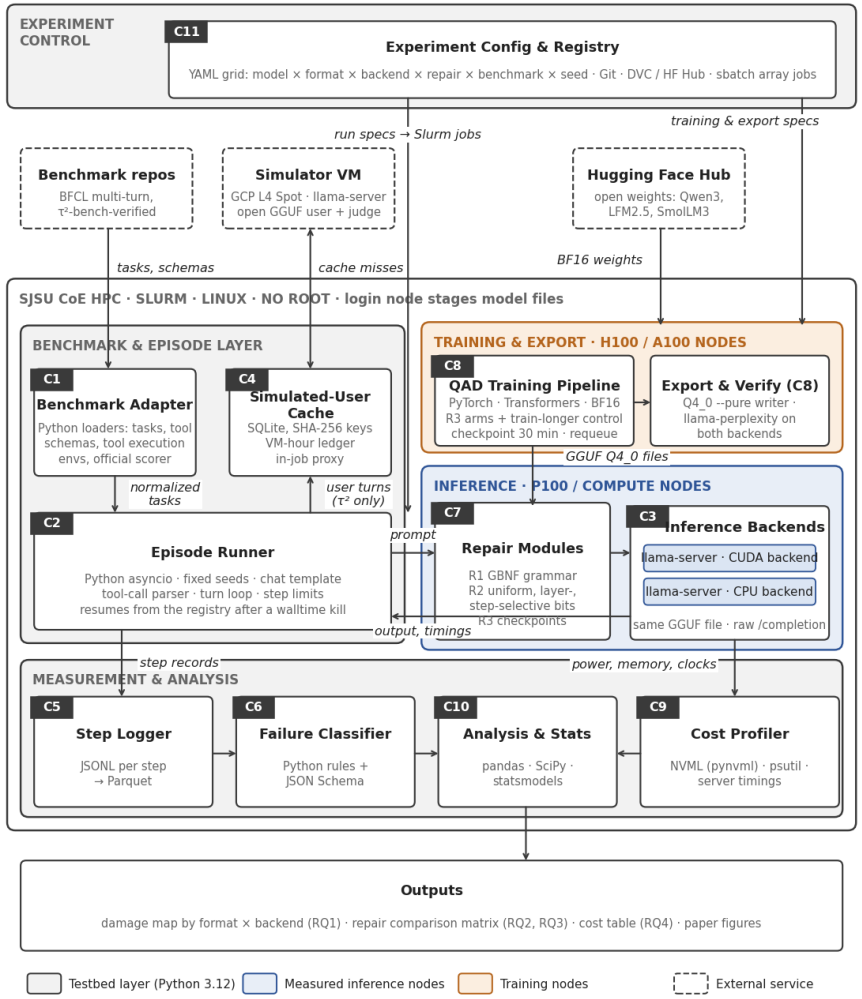
RQ1 Where does 4-bit damage concentrate (step type, depth)?

RQ2 How do the three repairs compare, each with its control?

RQ3 Do repairs fix the failures or only the score?

RQ4 What does each repair cost to deploy?

Testbed architecture



- Pipeline: config → episodes → step logs → failure labels → statistics
- Pluggable inference: llama.cpp CUDA and CPU backends on the same GGUF file; repairs are hooks
- Every agent step logged and labeled with one of 10 failure types

Testbed & runtime runner, backends, logger, R1, cost

Benchmarks & analysis loaders, sim cache, classifier, stats

Models & repairs R2, QAD, conversion, registry

Study design

Axis	Values
Models	Qwen3 0.6B / 1.7B / 4B · LFM2.5 1.2B / 2.6B · SmolLM3-3B
Formats	BF16 reference · Q4_0 · Q4_K_M · IQ4_XS on CUDA; Q4_0 also on CPU
Repairs (control)	R1 grammar (no grammar; grammar at 16-bit) · R2 Q5_0/Q6_K/Q8_0, 5.5-bit layer-selective, two-copy step routing (Q4_0) · R3 QAD-KL, QAD-KL+CE (train-longer at full precision)
Benchmarks	BFCL multi-turn (300) · τ^2-bench-verified (278) · BFCL single-turn (1,000)
Metrics	Task success, pass^k, failures per type, amplification ratio, memory / TTFT / tok/s / energy

30 functional + 11 non-functional requirements, each with an acceptance test · analysis pre-registered on OSF at M2 (Nov 6)

Progress, Sprints 1–3 (to Oct 9)

Sprint	Testbed & runtime	Benchmarks & analysis	Models & repairs
S1 Aug 31–Sep 13	Reading list: runtimes, GGUF, constrained decoding (T2)	Reading list: benchmarks, evaluation (T3) Team-formation slides (A2) Abstract rework and class talk, joint (A1, T1)	Abstract rework and class talk, joint; advisor-approved Sep 3 (A1, T1) Reading list: mixed precision, QAD (T4) Problem statement + plan draft (T5)
S2 Sep 14–27	GitHub repo + Projects board (T6) WB1 Ch5, Ch8 (T10) M1–M8 plan + Gantt, joint (A6) *Laptop llama.cpp CPU build (T11, in progress)*	WB1 Ch3 requirements (T8) Refined abstract (A7) *bfcl-eval + τ^2-bench setup (T12, in progress)*	RQ1–RQ4 problem statement (A5) M1–M8 plan + Gantt, joint (A6, T14) WB1 Ch2, Ch4, Ch9 (T9) *Repo layout + Dockerfile (T13, in progress)*
S3 Sep 28–Oct 11	Compute sizing + HPC request, joint (T15, A9)	Methodology, Ch3 + Ch7 (A8) Analysis policy reconciled (T16) Advisor review arranged (T18)	References re-verified (T17) HPC request, joint (A9) **First NVIDIA pilot runs, Oct 8**

Team Ch1 literature review, 117 references (T7) · WB1 submitted: 83 pp, 40 tables, 6 figures · 7 meetings logged

Open S2/S4 logs due Oct 9 (T19) · GCP L4 quota (T23) · advisor meeting, week of Oct 12 (T22) · HPC pending since Sep 30

Source: team meeting log in the team GitHub repo (/meetings); IDs in brackets; italic = in progress; column colors match slide 1

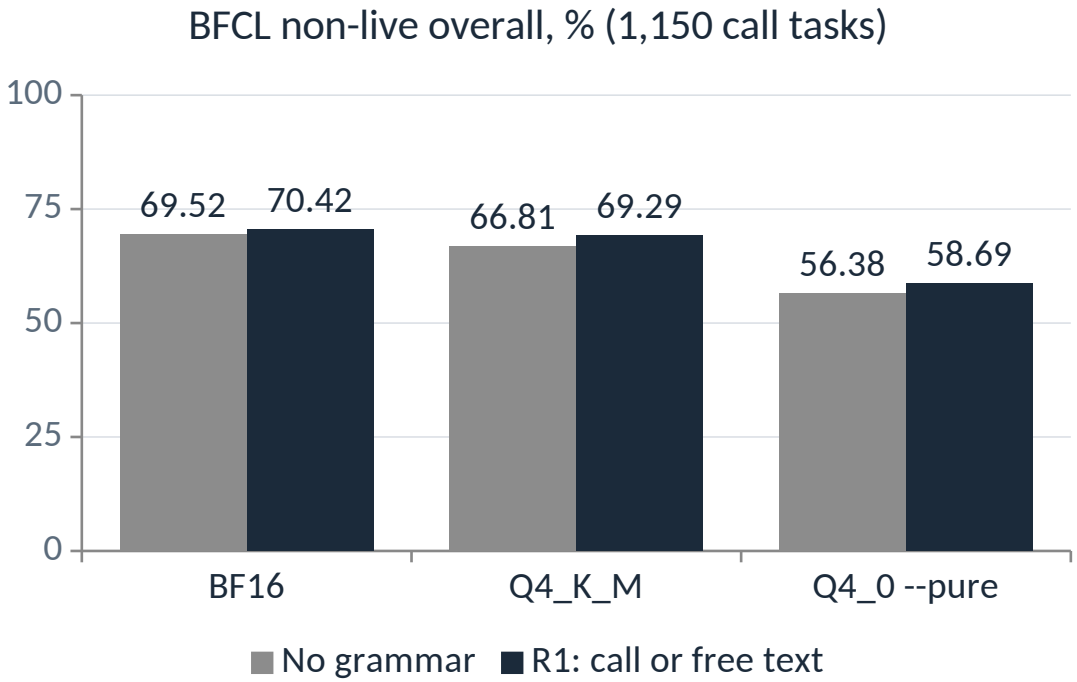
First NVIDIA pilot runs (Oct 8)

Format	Bits / weight	Decode tok/s	Peak GPU MiB	mJ / token
BF16	16.06	291.5	3096	349
Q4_K_M	5.15	342.2	2316	185
Q4_0 --pure	4.57	379.3	2262	134

Q4_K_M: small but detectable drop (non-live overall 66.8 vs 69.5%; p = 0.03, 1,150 paired tasks); published collapse not reproduced

Q4_0 --pure −13.1 points non-live: mostly skipped calls (57 vs 11 of 400 simple tasks)

Grammar, free text allowed (our R1): +2.5 / +2.3 points non-live; Q4_K_M 69.29 vs BF16 69.52%. Forcing a call breaks irrelevance



4 sampled runs, no grammar: 68.73 ± 0.55 · 65.32 ± 0.99 · 56.58 ± 1.65
Forced-call ablation on irrelevance (n = 240): 0.42 · 0.42 · 0.00%
Order: BF16 · Q4_K_M · Q4_0. Greedy, prompt cache off; irrelevance tasks need no call.

Pilot / development data, excluded from the study. Run on a Colab A100, Oct 8, by the C3/R1 backup owner while HPC access is pending (DEP-4); the S4 and S6 runs are repeated by their owners.

What could stop us

DEP-4 Pending

HPC accounts pending since Sep 30

Fallback: Colab A100 in use now (slide 7); laptops run the walking skeleton

DEP-8 High for 0.6B

Qwen3-0.6B solves too few multi-turn tasks

Fallback: add the BFCL single-turn set; a model under 120 solvable tasks gets single-turn analysis only

DEP-3 Medium

L4 Spot VM for τ^2-bench's simulated user (~$130) not secured yet

Fallback: on-demand L4; cluster A100; BFCL alone still covers RQ1–RQ3

DEP-9 Medium

Someone publishes the comparison first

Fallback: OSF timestamp at M2; our controls and measured costs stay new

Next three weeks

Today, Oct 9

M1 · Oct 23
Walking skeleton (local=true fallback)

M2 · Nov 6
BF16 references; analysis plan frozen on OSF

M3 · Dec 4
Damage map v1 (RQ1)

M4 · Dec 18
R1 + R2 evaluated, end of 295A

Before M1, by work stream

Testbed & runtime — Rerun S1/S3/S6 · C2 runner, C3 backends, C5 logger · R1, C9 profiler, CI

Benchmarks & analysis — S4 multi-turn + C1 loaders · S5 simulator · C4 cache, C6 rules v0, C10 toy stats

Models & repairs — S8 export · C11 registry + GCP quota · R2 Q8_0 + S9 switch · Dockerfile

Deliverables by M4 DL-1 testbed v0.1 · DL-2 codebook + DL-3 OSF plan (M2) · DL-4 damage map (M3) · DL-5 R1/R2

Deliverables

Backup

ID	Deliverable	Target
DL-1	Open-source testbed, C1–C11	v0.1 at M4; v1.0 + DOI at M8
DL-2	Failure taxonomy, classifier, κ report	v0 at M2; v1 at M3
DL-3	Pre-registered analysis plan (OSF)	M2
DL-4	Damage map (RQ1)	v1 at M3; final at M8
DL-5	Repair comparison matrix (RQ2, RQ3)	R1, R2 at M4; R3 at M5; costs at M6
DL-6	QAD pipeline with exact Q4_0 export	M5
DL-7	Cost measurements (RQ4)	M6
DL-8	Step-log dataset (Hugging Face, Zenodo DOI)	v1 after M7; final at M8
DL-9	Workshop paper (EuroMLSys 2027)	M7

Also AD-5 tool-schema-to-grammar compiler (M4) · AD-3 public damage-map report · AD-1 retrained checkpoints on Hugging Face

Metrics and hypotheses

Backup

Key metrics

- Task success, pass^k
- Step failures by type
- Amplification ratio vs 16-bit
- Flip rate
- Gap closed by a repair
- Memory, TTFT, tok/s, energy

H1 (RQ1) 4-bit raises per-step failures above 16-bit; the excess concentrates in tool-call steps and at depth

H2 (RQ2) Each repair cuts excess failures vs its control; R1 fixes format failures, not semantic ones; QAD beats train-longer

H3 (RQ3) Repairs close more of the score gap than of the failure gap

H4 (RQ4) R1 adds no weight memory and at most 5% to median per-step decode latency on CUDA

State of the art in industry

Backup

Vendor	Release	Technique	Target
Meta	Llama 3.2 1B/3B, Oct 2024	QAT with LoRA; SpinQuant	4-bit weights, 8-bit activations
Google	Gemma 3 QAT, Apr 2025	QAT toward the unquantized model	Q4_0 for llama.cpp, Ollama, MLX
Apple	On-device model, 2025	2-bit QAT + LoRA recovery adapters	Apple silicon
NVIDIA	NVFP4 QAD, 2026	KL distillation from BF16 teacher	NVFP4 GPUs
Google	Gemma 4 QAT, Jun 2026	QAT; Q4_0 plus a mobile format	E2B at about 1 GB
Multiverse	Quantization-aware healing, 2026	Distillation into a 4-bit model	MXFP4
Liquid AI	LFM2.5 QAD, Aug 2026	QAD for 230M to 2.6B models	GGUF Q4_0

Structured output ships in OpenAI, vLLM/SGLang (XGrammar) and llama.cpp (GBNF); these engines are judged on validity and speed.
No vendor reports per-failure-type agent results for its quantized builds or compares the three repairs.

Work streams and backups

Backup

Work stream	Owns	Key deliverables	Backup for
Testbed & runtime	C2 runner, C3 backends, C5 logger, R1 grammar, C9 cost profiler	Testbed core on CUDA + CPU; cluster builds, Slurm runner; v0.1 + CI; R1 grammars; RQ4 cost matrix; Expo demo	C1, C4, R2, C8
Benchmarks & analysis	C1 benchmark adapter, C4 simulator cache, C6 failure classifier, C10 analysis + stats	Inclusion set; simulator + cache; codebook + labeled sample; OSF pre-registration; damage map (RQ1); RQ3 analysis	C2, C5, C9, C11
Models & repairs	R2 added bits, R3 + C8 QAD pipeline, conversion + 4-bit checks, C11 config + registry	R2 variants; QAD checkpoints on the Hub; verified GGUF exports; lockfile, Dockerfile, Singularity; v1.0 + DOI	C3, C6, R1, C10

Workbook rule: “A spike is done only when the component’s backup owner reruns it under their own cluster account.” The Oct 8 Colab runs are development data and close no spike.

Full schedule

Backup



Workbook Fig 6. Bar colors match the name tags on slide 1; gray = shared work; diamonds = milestones M1–M8; shaded band = winter break.